

LAB 01:

The screenshot shows the pgAdmin 4 interface with a SQL query and its execution plan. The query is as follows:

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT
3   c.first_name,
4   c.last_name,
5   r.rental_date
6 FROM customer c
7 JOIN rental r
8   ON c.customer_id = r.customer_id
9 WHERE c.customer_id = 1;
```

The execution plan shows the following steps:

- 1 Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=1.349..5.560 rows=32.00 loops=1)
- 2 Buffers: shared read=153
- 3 -> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=0.256..0.259 rows=1.00 loops=1)
- 4 Index Cond: (customer_id = 1)
- 5 Index Searches: 1
- 6 Buffers: shared read=3
- 7 -> Seq Scan on rental r (cost=0.00..350.55 rows=32 width=10) (actual time=1.089..5.288 rows=32.00 loops=1)
- 8 Filter: (customer_id = 1)
- 9 Rows Removed by Filter: 16012
- 10 Buffers: shared read=150
- 11 Planning:
- 12 Buffers: shared hit=140 read=12 dirtied=3
- 13 Planning Time: 11.059 ms

Total rows: 14 Query complete 00:00:00.100

1, The **customer** table is the Outer table, while the **rental** table is the Inner table. The database first finds customer ID 1 and then searches the rental table for matching rental records.

2, The customer table returns only one row because of **WHERE c.customer_id = 1**. Since the Outer relation is extremely small, a Nested Loop is efficient: PostgreSQL only needs to take this one customer and look up the matching rental records. Building a hash table for a Hash Join would introduce unnecessary overhead.

3, The Inner table has **loops = 1** because the Outer **customer** table returns only one row. In a Nested Loop, the Inner operation is executed once for each Outer row. Therefore, one Outer row results in one Inner loop.

LAB 02:

The screenshot shows the pgAdmin 4 interface with a query window open. The query is as follows:

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT
3     f.title,
4     l.name AS language
5 FROM film f
6 JOIN language l
7     ON f.language_id = l.language_id;
```

Below the query, the 'Data Output' tab displays the 'QUERY PLAN' for the executed query. The plan details are as follows:

Step	Operation	Cost	Actual Time	Actual Rows	Width	Loops
1	Hash Join	(cost=1.14..103.62 rows=1000 width=99)	(actual time=0.052..0.433 rows=1000.00 loops=1)			
2	Hash Cond: (f.language_id = l.language_id)					
3	Buffers: shared hit=89					
4	-> Seq Scan on film f	(cost=0.00..98.00 rows=1000 width=17)	(actual time=0.028..0.177 rows=1000.00 loops=1)			
5	Buffers: shared hit=88					
6	-> Hash	(cost=1.06..1.06 rows=6 width=88)	(actual time=0.016..0.016 rows=6.00 loops=1)			
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB					
8	Buffers: shared hit=1					
9	-> Seq Scan on language l	(cost=0.00..1.06 rows=6 width=88)	(actual time=0.008..0.009 rows=6.00 loops=1)			
10	Buffers: shared hit=1					
11	Planning Time: 0.221 ms					
12	Execution Time: 0.505 ms					

At the bottom of the query plan, it states: 'Total rows: 12 Query complete 00:00:00.095'. The status bar at the bottom of the pgAdmin window shows 'CRLF Ln 7, Col 38'.

1, The **language** table is the Build side of the Hash Join. PostgreSQL scans its 6 rows and builds a hash table using **language_id**. The **film** table is the Probe side, where all 1,000 film rows are scanned and matched against the hash table.

2, The Hash node uses 9 kB of memory. Since **Batches = 1**, the entire hash table fits in memory and PostgreSQL does not need to spill any part of it to disk.

3, After setting **work_mem** to 64 kB, I expect little or no change because the hash table only requires 9 kB, which is well below the 64 kB limit. Therefore, the hash table should still fit entirely in memory, **Batches** should remain 1, and PostgreSQL should continue using a Hash Join.

LAB 03:

The screenshot shows the pgAdmin 4 interface with a SQL query executed. The query is as follows:

```
5 SELECT
6   f.film_id,
7   f.title,
8   i.inventory_id
9 FROM film f
10 JOIN inventory i
11   ON f.film_id = i.film_id
12 ORDER BY f.film_id;
```

The Query Plan shows the following steps:

- 1 Merge Join (cost=349.64..547.91 rows=4581 width=23) (actual time=1.508..3.478 rows=4581.00 loops=1)
- 2 Merge Cond: (f.film_id = i.film_id)
- 3 Buffers: shared hit=109
- 4 -> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.014..0.469 rows=1000.00 loops=1)
- 5 Index Searches: 1
- 6 Buffers: shared hit=84
- 7 -> Sort (cost=349.37..360.82 rows=4581 width=6) (actual time=1.487..1.783 rows=4581.00 loops=1)
- 8 Sort Key: i.film_id
- 9 Sort Method: quicksort Memory: 300kB
- 10 Buffers: shared hit=25
- 11 -> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.024..0.673 rows=4581.00 loops=1)
- 12 Buffers: shared hit=25
- 13 Planning:
- 14 Buffers: shared hit=6

The query completed successfully, showing 16 rows affected. The total query runtime was 100 msec.

1, The **film** table uses an Index Scan on **film_pkey**, which already returns rows ordered by **film_id**. Therefore, no explicit Sort is needed for **film**. The **inventory** table uses a Sequential Scan, followed by a Sort on **i.film_id**. The sort uses quicksort and 300 kB of memory.

2, After **RESET ALL** and removing **ORDER BY**, PostgreSQL may switch back to a Hash Join because sorted output is no longer required. The Hash Join avoids the additional sorting cost and may therefore have a lower estimated execution cost.

LAB HANDS-ON:

Task 1:

The screenshot shows the pgAdmin 4 interface with a query window open. The query is:

```
1 SET enable_indexscan = OFF;  
2  
3 EXPLAIN (ANALYZE, BUFFERS)  
4 SELECT *  
5 FROM rental  
6 WHERE rental_date > '2005-05-24';
```

The query has been executed, and the execution plan is displayed in the 'Data Output' tab. The plan shows a sequential scan on the 'rental' table.

QUERY PLAN
Seq Scan on rental (cost=0.00..350.55 rows=16044 width=36) (actual time=0.017..1.034 rows=16044.00 loops=1)
Filter: (rental_date > '2005-05-24 00:00:00'::timestamp without time zone)
Buffers: shared hit=150
Planning:
Buffers: shared hit=85 read=3 dirtied=2
Planning Time: 9.231 ms
Execution Time: 1.379 ms

Total rows: 7 Query complete 00:00:00.006

The screenshot shows the pgAdmin 4 interface with a query window open. The query is:

```
1 SET enable_indexscan = ON;  
2 SET enable_bitmapscan = ON;  
3  
4 EXPLAIN (ANALYZE, BUFFERS)  
5 SELECT *  
6 FROM rental  
7 WHERE rental_date > '2005-05-24';
```

The query has been executed, and the execution plan is displayed in the 'Data Output' tab. The plan shows a sequential scan on the 'rental' table.

QUERY PLAN
Seq Scan on rental (cost=0.00..350.55 rows=16044 width=36) (actual time=0.013..1.549 rows=16044.00 loops=1)
Filter: (rental_date > '2005-05-24 00:00:00'::timestamp without time zone)
Buffers: shared hit=150
Planning:
Buffers: shared hit=3
Planning Time: 0.174 ms
Execution Time: 1.890 ms

Total rows: 7 Query complete 00:00:00.069

Khi disable Index Scan

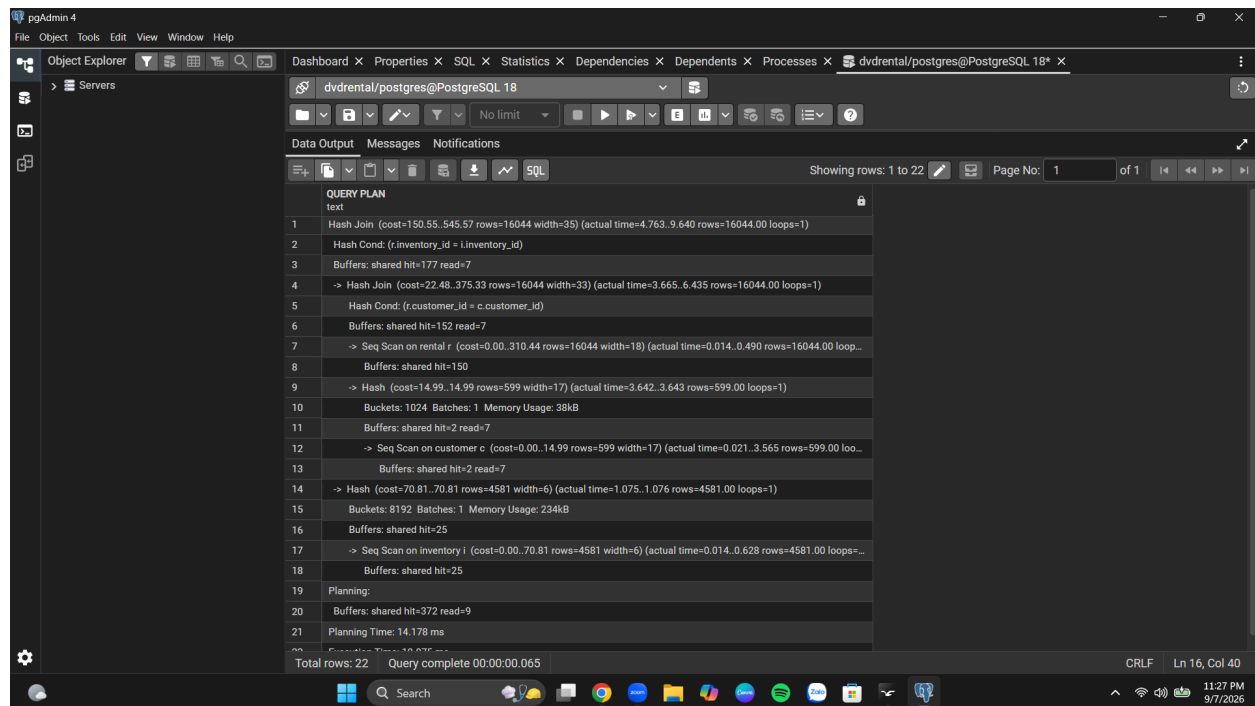
With `enable_indexscan = OFF`, PostgreSQL used a Sequential Scan on the `rental` table. The query returned 16,044 rows, with an execution time of 1.379 ms.

Khi re-enable Index Scan

After re-enabling Index Scan and Bitmap Scan, PostgreSQL still selected a Sequential Scan. The estimated cost remained `0.00.350.55`, and the query again returned 16,044 rows.

Re-enabling Index Scan does not force PostgreSQL to use an index. The condition `rental_date > '2005-05-24'` returns a very large proportion of the `rental` table. Because many rows must be retrieved, scanning the table sequentially is cheaper than using an index and performing many table lookups. Therefore, the optimizer correctly keeps the Sequential Scan.

Task 2:



The screenshot displays the pgAdmin 4 interface with the Query Plan for a three-way join query. The plan is as follows:

- 1 Hash Join (cost=150.55..545.57 rows=16044 width=35) (actual time=4.763..9.640 rows=16044.00 loops=1)
- 2 Hash Cond: (r.inventory_id = i.inventory_id)
- 3 Buffers: shared hit=177 read=7
- 4 -> Hash Join (cost=22.48..375.33 rows=16044 width=33) (actual time=3.665..6.435 rows=16044.00 loops=1)
- 5 Hash Cond: (r.customer_id = c.customer_id)
- 6 Buffers: shared hit=152 read=7
- 7 -> Seq Scan on rental r (cost=0.00..310.44 rows=16044 width=18) (actual time=0.014..0.490 rows=16044.00 loops=1)
- 8 Buffers: shared hit=150
- 9 -> Hash (cost=14.99..14.99 rows=599 width=17) (actual time=3.642..3.643 rows=599.00 loops=1)
- 10 Buckets: 1024 Batches: 1 Memory Usage: 38kB
- 11 Buffers: shared hit=2 read=7
- 12 -> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=17) (actual time=0.021..0.356 rows=599.00 loops=1)
- 13 Buffers: shared hit=2 read=7
- 14 -> Hash (cost=70.81..70.81 rows=4581 width=6) (actual time=1.075..1.076 rows=4581.00 loops=1)
- 15 Buckets: 8192 Batches: 1 Memory Usage: 234kB
- 16 Buffers: shared hit=25
- 17 -> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.014..0.628 rows=4581.00 loops=1)
- 18 Buffers: shared hit=25
- 19 Planning:
- 20 Buffers: shared hit=372 read=9
- 21 Planning Time: 14.178 ms

Total rows: 22 Query complete 00:00:00.065

PostgreSQL selected two Hash Joins for the three-way join. In the first join, the `customer` table is the Build side with 599 rows, while `rental` is the Probe side with 16,044 rows. The customer hash table uses 38 kB of memory. The resulting 16,044 rows are then joined with `inventory`. In the second Hash Join, PostgreSQL builds a hash table from the 4,581 inventory rows using 234 kB of memory and probes it with the intermediate result. Both hash operations use `Batches = 1`, indicating that the hash tables fit entirely in memory. Hash Join is appropriate because the query contains unrestricted equality joins and processes a large proportion of each table.

Challenge:

The screenshot shows the pgAdmin 4 interface with a query execution plan for a PostgreSQL database. The plan is displayed in a table format with 22 rows. The first two rows are highlighted in blue, indicating the Hash Joins. The statistics for the first Hash Join are: cost=150.55..545.57, rows=16044 width=35, actual time=0.781..7.021 rows=16044.00, loops=1. The second Hash Join has: cost=22.48..375.33 rows=16044 width=33, actual time=0.190..4.187 rows=16044.00, loops=1. The plan also shows sequential scans on rental, customer, and inventory tables, and various buffer and planning statistics.

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Loops
1	Hash Join	cost=150.55..545.57	rows=16044 width=35		actual time=0.781..7.021	rows=16044.00	loops=1
2	Hash Cond: (r.inventory_id = i.inventory_id)						
3	Buffers: shared hit=184						
4	-> Hash Join	cost=22.48..375.33	rows=16044 width=33		actual time=0.190..4.187	rows=16044.00	loops=1
5	Hash Cond: (r.customer_id = c.customer_id)						
6	Buffers: shared hit=159						
7	-> Seq Scan on rental r	cost=0.00..310.44	rows=16044 width=18		actual time=0.018..0.511	rows=16044.00	loops=1
8	Buffers: shared hit=150						
9	-> Hash	cost=14.99..14.99	rows=599 width=17		actual time=0.162..0.163	rows=599.00	loops=1
10	Buckets: 1024 Batches: 1 Memory Usage: 38kB						
11	Buffers: shared hit=9						
12	-> Seq Scan on customer c	cost=0.00..14.99	rows=599 width=17		actual time=0.009..0.080	rows=599.00	loops=1
13	Buffers: shared hit=9						
14	-> Hash	cost=70.81..70.81	rows=4581 width=6		actual time=0.564..0.565	rows=4581.00	loops=1
15	Buckets: 8192 Batches: 1 Memory Usage: 234kB						
16	Buffers: shared hit=25						
17	-> Seq Scan on inventory i	cost=0.00..70.81	rows=4581 width=6		actual time=0.006..0.248	rows=4581.00	loops=1
18	Buffers: shared hit=25						
19	Planning:						
20	Buffers: shared hit=63						
21	Planning Time: 0.470 ms						
22	Execution Time: 7.427 ms						

Total rows: 22 Query complete 00:00:00.097

The execution plan did not change after **ANALYZE**. PostgreSQL still selected two Hash Joins because the updated statistics confirmed that this remained the lowest-cost strategy. The estimated and actual row counts were highly consistent, showing that the optimizer had accurate cardinality estimates.